

OpenDecoder: Open Large Language Model Decoding to Incorporate Document Quality in RAG

Fengran Mo
Université de Montréal
Montréal, Québec, Canada
fengran.mo@umontreal.ca

Zhan Su
Université de Montréal
Montréal, Québec, Canada
zhan.su@umontreal.ca

Yuchen Hui
Université de Montréal
Montréal, Québec, Canada
yuchen.hui@umontreal.ca

Jinghan Zhang
Clemson University
Clemson, South Carolina, USA
jinghaz@clemson.edu

Jia Ao Sun
Université de Montréal
Montréal, Québec, Canada
jia.ao.sun@umontreal.ca

Zheyuan Liu
University of Notre Dame
Notre Dame, Indiana, USA
zliu29@nd.edu

Chao Zhang
Georgia Institute of Technology
Atlanta, Georgia, USA
chaozhang@gatech.edu

Tetsuya Sakai
Waseda University
Tokyo, Japan
tetsuya@waseda.jp

Jian-Yun Nie
Université de Montréal
Montréal, Québec, Canada
nie@iro.umontreal.ca

Abstract

The development of large language models (LLMs) has achieved superior performance in a range of downstream tasks, including LLM-based retrieval-augmented generation (RAG). The quality of generated content heavily relies on the usefulness of the retrieved information and the capacity of LLMs' internal information processing mechanism to incorporate it in answer generation. It is generally assumed that the retrieved information is relevant to the question. However, the retrieved information may have a variable degree of relevance and usefulness, depending on the question and the document collection. It is important to take into account the relevance of the retrieved information in answer generation. In this paper, we propose *OpenDecoder*, a new approach that leverages explicit evaluation of the retrieved information as quality indicator features for generation. We aim to build a RAG model that is more robust to varying levels of noisy context. Three types of explicit evaluation information are considered: relevance score, ranking score, and QPP (query performance prediction) score. The experimental results on five benchmark datasets demonstrate the effectiveness and better robustness of *OpenDecoder* by outperforming various baseline methods. Importantly, this paradigm is flexible to be integrated with the post-training of LLMs for any purposes and incorporated with any type of external indicators.

CCS Concepts

• **Information systems** → **Information retrieval**; **Data management systems**; • **Computing methodologies** → **Artificial intelligence**; **Machine learning**.

Keywords

Information Retrieval, Retrieval-Augmented Generation, Robust Question Answer, Decoding, Large Language Model

ACM Reference Format:

Fengran Mo, Zhan Su, Yuchen Hui, Jinghan Zhang, Jia Ao Sun, Zheyuan Liu, Chao Zhang, Tetsuya Sakai, and Jian-Yun Nie. 2026. OpenDecoder: Open Large Language Model Decoding to Incorporate Document Quality in RAG. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3774904.3792524>

1 Introduction

The development of large language models (LLMs) [1, 45, 62] has achieved superior performance in a range of downstream tasks via their parametric knowledge acquisition from the training documents. However, LLMs still encounter foundational problems, such as understanding the limits of their knowledge and capability [13, 51], where a lack of sufficient knowledge might lead to hallucinations or generating outdated results [16, 26]. Retrieval-augmented generation (RAG) [25] is a common practice to address the incomplete knowledge issue by incorporating external information to obtain more accurate and reliable content generation.

Despite the fact that the RAG technique alleviates the knowledge boundary issue of LLMs, existing approaches to RAG face fundamental challenges: the quality of generated content heavily relies on the usefulness of the retrieved information and the capacity of LLMs' internal information processing mechanism [28, 41]. It is generally assumed that the retrieved information is relevant and useful for content generation, or LLMs have the capability to judge its relevance. However, the existing literature [9] showed the vulnerability of automated usefulness-checking systems when confronted with noisy information. Thus, the defective and imperfect retrieved information would degrade the performance of LLMs. As a matter of fact, when an LLM is asked to answer a question based on an irrelevant document, the quality of the answer is negatively affected [46]. Such a situation with irrelevant information



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792524>

may often occur when RAG is asked to deal with a large variety of questions. An ideal RAG system should be able to understand and tolerate the noisy input, i.e., process the diverse inputs that include useful evidence and irrelevant information, without being affected by the noise and resulting in significant degradation in performance [40, 65]. For example, if the input context is partially noisy or extremely irrelevant, the system can attend only to the useful part or ignore the whole misinformation when generating an answer.

Existing studies attempt to address this issue from various perspectives, which can be categorized into (i) workflow-based methods and (ii) fine-tuning-based methods. The first category aims to design a workflow that navigates LLMs to identify useful pieces from retrieved information and append them to the final input context for generation. The intermediate steps in the workflow vary and may include self-correction through LLM-as-a-judge [11, 58], isolating individual results for later aggregation [37, 54], and step-by-step filtering via reasoning [4], among others. This training-free approach is highly sensitive to the used prompt template and follows the strong assumption that the model could have enough capacity to distinguish the useful information by following the instruction to produce ideal output [13]. However, one cannot expect that LLMs always generate correct judgments, and thus the manipulated final input might lose crucial information or include wrong information before conducting answer generation [59]. Besides, the judgment workflow with multiple steps with LLM calling would significantly increase latency [38]. On the other hand, the fine-tuning methods aim to teach the model to incorporate external useful knowledge in an effective way. For example, one can equip the LLMs with retrieval defect detection and utility extraction via instruction fine-tuning [44, 46] or enable the LLMs to interact with the retriever multiple turns until appending sufficient information for answer generation [2, 18].

Though effective, the existing approaches still inherit the original method of LLMs to perform the online computation of key-value pairs in the attention networks of the decoder [41] for generation, which means that the autoregressive decoding of LLMs is mainly impacted by the attention score to produce generation probability. We notice that the attention score is assigned by LLMs alone once the retrieved documents are appended into the prompt template. The original relevance judged by the retriever of the input documents is never used by the LLMs. Thus, the LLMs might treat the input documents as equally relevant or slightly different according to their input position [23] based on the implicit internal judgments. This gives rise to several critical questions: *Should RAG ignore the relevance signals of the retrieved documents in its generation? Are such relevance signals useful for generation? How should the generation be impacted by document relevance?*

We believe that document relevance should be explicitly considered in answer generation in RAG, so that answer generation can be more tuned toward relevant information than irrelevant one. To achieve this goal, in this paper, we propose **OpenDecoder**, a new approach that directly leverages document relevance to change the information processing procedure of LLMs decoding, namely, its attention mechanism. As shown in Figure 1, compared to the current decoding paradigm of LLMs, our proposed **OpenDecoder** does not only rely on the attention score produced via the internal

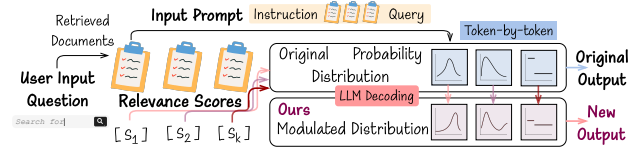


Figure 1: Comparison between the existing decoding LLMs that use their default probability distribution and our proposed approach that modifies the distribution by leveraging external explicit relevance signals.

network and instruction-following training, but also leverages explicit relevance signals as external indicator features. The model is expected to become more robust to varying levels of noisy input context by reshaping the generation probability distribution via the useful information among the retrieved knowledge, and thus produce more accurate answers as output.

To implement *OpenDecoder*, the first step is to construct external indicators by extracting quality features from the retrieved documents. We consider three types of signals: relevance score from the retriever, LLM-judged semantic score, and query performance prediction score. Then, we design a training framework to teach the LLMs to leverage these explicit indicator features (either separately or in combination) for answer decoding. Specifically, we incorporate the external features into the internal attention networks computation to directly modulate the LLMs when producing generation probabilities for the decoding candidate tokens. Additionally, to make the training and inference more robust to noisy information within the input, we conduct robustness training by reconstructing the input top-k documents via sampling additional documents with various relevant levels. During the online inference, the corresponding indicator features from external information are processed by the trained LLMs via the learned parameters in *OpenDecoder*. Experiments on five benchmark datasets covering both general and multi-hop question answering (QA) demonstrate the effectiveness and enhanced robustness of the proposed approach, which consistently outperforms the vanilla RAG and other strong baselines across diverse noisy environments. Importantly, our designed *OpenDecoder* is flexible to be integrated with the post-training of LLMs for any purposes and incorporate any other type of external indicator features towards effectiveness, robustness, or trustworthiness enhancement.

Our contributions are summarized as follows:

- (1) We propose a new approach *OpenDecoder* to directly modify the LLM decoding in RAG by leveraging the relevance signals of the retrieved documents.
- (2) We design a training method, which includes constructing explicit relevance indicators from retrieved documents, teaching the model to leverage explicit indicators for answer decoding, and improving robustness via replacing the original top-k documents with various relevant levels ones.
- (3) We conduct experiments on five widely used benchmarks, including general and multi-hop QA. Our *OpenDecoder* outperforms vanilla RAG and other strong baselines across diverse noisy environments, which demonstrates its superior effectiveness.

2 Related Work

2.1 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) [10, 25] aims to retrieve external resources to supplement LLMs to generate a response, showing significant advantages in knowledge-intensive tasks [8, 12, 20]. Earlier RAG methods follow the “Retrieve-then-Read” framework [15, 25] by adopting a retriever to search for relevant information from external resources based on the user’s query. To further enhance RAG performance, subsequent studies focus on refining retrieval quality through techniques such as query reformulation [31, 35], re-ranking [42, 59], and noise filtering as intermediate steps [17, 37], thereby improving the relevance of documents before they are appended to LLMs’ input.

However, retrieval errors remain common due to limitations in search effectiveness and corpus quality [36], which can ultimately degrade RAG performance. To address this problem, robust RAG [30, 64] focuses on input optimization and knowledge integration. For instance, Weller et al. [53] conduct query augmentation and introduce a novel confidence method based on answer redundancy. RobustRAG [54] employs an isolate-then-aggregate strategy to ensure the robustness of LLM responses against retrieval corruption attacks. By generating self-synthesized rationales, InstructRAG [52] explicitly denoises the retrieved content, thereby enhancing the robustness of RAG systems. AstuteRAG [48] turns to refine and integrate knowledge derived from different sources to improve knowledge utilization and enhance the robustness of the generated answer. RbFT [46] proposes a robust fine-tuning strategy against retrieval defects with two defined tasks, defect detection and utility extraction, with associated instructions. In addition, recent studies on developing deep search agents [18, 27, 39, 63] introduce a new paradigm for enhancing input quality by integrating in-context reasoning with dynamic search tool invocation when needed. Although effective, these existing methods rely only on the internal mechanism of LLMs to process information, e.g., attention network [47]. Unlike them, our method *OpenDecoder* is developed to enable LLMs to distinguish useful information via both internal mechanisms and external explicit indicators.

2.2 Decoding Optimization in LLMs

Prompting [29] the advanced LLMs is a simple and effective way to instruct them to generate answers, where the answer decoding highly relies on the designed prompt and internal attention mechanism. Existing literature optimizes the decoding procedure of LLMs on various aspects. For efficiency, Performers [5] propose compressed attention, reducing attention complexity from quadratic to linear. StreamingLLM [55] leverages attention sinks to decrease Key-Value cache memory for long-context generation. For effectiveness, a series of studies [3, 43, 60] investigate how to leverage inference scaling and deep reasoning for RAG decoding. A recent study REFRAG [28] rethinks RAG-based decoding and proposes an optimized architecture to compress only a small subset of retrieved documents that are directly related to the query for effective and efficient decoding. For faithfulness, the existing studies aim to detect and manage misinformation within retrieved documents [65], such as explicitly identifying and resolving knowledge conflicts [7, 49, 61]. These studies focus on selecting relevant

and reliable information for LLM input, which still operate in the way that has been trained, by assuming the input information to be relevant. In contrast, in our approach, we modify the attention mechanism according to the relevance of retrieved information. Such an approach has not been proposed in the literature.

3 OpenDecoder

The principle of our methodology *OpenDecoder* is to modify the decoding procedure of LLMs with explicit relevance information as quality indicators, rather than solely based on prompt design. The goal is to enable the model to be robust to noisy retrieved information that can be irrelevant. In the following sections, we first formulate the problem and provide an overview of our *OpenDecoder* as shown in Figure 2. Then, we present the detailed design for the components in *OpenDecoder*, including (1) constructing quality indicators via extracting features from external information; (2) learning to leverage explicit indicators for decoding; and (3) robustness training via replacing the input retrieved documents with various relevant levels.

3.1 Task Formulation

A vanilla RAG system typically consists of an ad-hoc retriever $\mathcal{R}$, a generator (i.e., the LLM) $\mathcal{G}$, and a corresponding corpus $\mathcal{C}$ with a large collection of documents. Given a user query q , the retriever $\mathcal{R}$ would identify its top- k relevant documents $\mathcal{R}(q) = \{\text{doc}_i^q\}_{i=1}^k$. Then, the LLM $\mathcal{G}$ would generate an answer a based on the query and relevant documents as

$$a = \mathcal{G}(q, \{\text{doc}_i^q\}_{i=1}^k) = \mathcal{G}(q, \mathcal{R}(q, \mathcal{C})) \quad (1)$$

The quality of the generated answer a highly depends on the useful information returned by the retriever $\mathcal{R}$ and the understanding capacity of LLMs for the input context with the corresponding prompt. The inevitable noise in the retrieved context would significantly degrade the answer quality of LLMs on top of it. These issues are unavoidable with the current prompting-based approach, where the content decoding only inherits the internal information processing mechanism of LLMs by following the prompt instruction [13, 51]. Our work focuses on guiding the decoding processing with explicit signals of external indicators of usefulness beyond the scores produced by the internal attention network.

3.2 Constructing Indicators via Extracting Features from External Information

Our goal is to incorporate external explicit indicators for LLMs to utilize internal knowledge stored in their parameters. Thus, the first step is to construct the indicators by extracting quality features from the retrieved information. The most intuitive feature is the relevant score computed by the retriever model in terms of the given query and candidate documents. In general, the retrieved top- k relevant documents $\{\text{doc}_i^q\}_{i=1}^k$ for the query q are associated with their relevance scores $\mathcal{S}^{\text{Ret}} = \{s_i^{\text{Ret}}\}_{i=1}^k$, each computed by a similarity function as $s_i^{\text{Ret}} = \frac{q \cdot \text{doc}_i^q}{\|q\| \|\text{doc}_i^q\|}$. Since external indicators can be constructed in multiple ways, different features may be extracted and computed depending on the specific requirements, such as for faithfulness or trustworthiness.

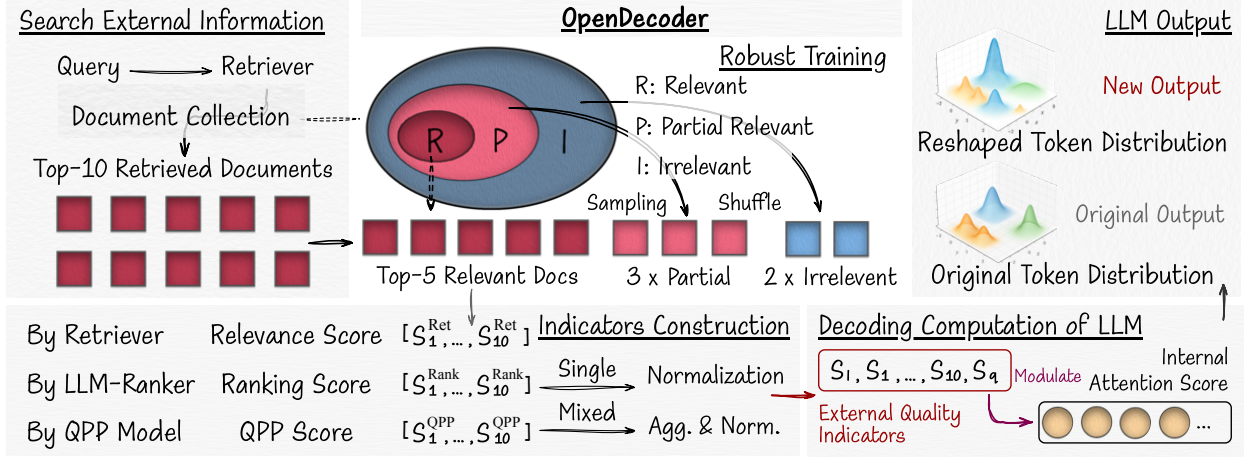


Figure 2: The framework of *OpenDecoder*, including Searching External Information with top-k retrieved documents, Indicators Construction based on the retrieved documents with various types of quality scores, teaching the model to leverage external explicit quality indicators for the Decoding Computation of LLM by modulating internal attention score computation and applying Robust Training, and finally obtaining the reshaped token probability distribution during content generation.

In our implementation, we further leverage two additional indicators features, (i) the relevance judged by a LLM-based ranker as $S^{\text{Rank}} = \{s_i^{\text{Rank}}\}_{i=1}^k$; and (ii) the query performance prediction (QPP) score S^{QPP} judged by a QPP model [34]. Specifically, we use the logit of the end-of-sequence token for the LLM-ranker judged score as $s_i^{\text{Rank}} = \text{Ranker}(q, \text{doc}_i^q)[-1]$ following [32], and the logit of token “relevant” in the prediction of the QPP model for each given document doc_i^q in the candidate list as $s_i^{\text{QPP}} = \text{logit}(\text{“relevant”} | (q, \text{doc}_i^q))$. The relevance judged by the LLM-based ranker is expected to provide semantic similarity features from another perspective and help to investigate whether these explicit LLM-judged signals have additional impacts or have been integrated in model internal processing implicitly. Besides, the QPP scores provide the indicators about the difficulty of the query, which might imply the possible noisy level of the retrieved information for the generator.

Eventually, these scores calculated based on different aspects are used individually or as a combination S^{agg} by an aggregation function to guide the LLMs to process the external information during generation, i.e., to decide to what extent it should focus on different parts of the input context in decoding.

3.3 Learning to Leverage Explicit Indicators Features for Decoding

The fundamental problem in the current paradigm of RAG is that adding external retrieved information in the input prompt could only affect the online computation of key-value pairs in the attention networks of LLMs, which is not tailored to the input with noise. Since the retrieved context is usually not perfect, the inherent defects are only implicitly processed via the attention score computation, which is influenced by the mechanisms (e.g., predefined system prompt) in the pre-training procedure. Thus, a better way is to inform the decoding with additional explicit indicators directly,

so that the LLMs know how much they should rely on external or internal knowledge to generate an answer.

To this end, we aim to teach the model to leverage the explicit indicator features from external information generated in Sec. 3.2, and integrate them into the original attention networks computation. Following the procedure of the standard RAG, the user query q and its corresponding retrieved top-k documents $\mathcal{R}(q) = \{\text{doc}_i^q\}_{i=1}^k$ would fill the prompt template together with the instruction as $[\text{Instruction}, \text{doc}_1^q, \text{doc}_2^q, \dots, \text{doc}_k^q, \text{query}]$ to instruct the LLM to produce an answer. To teach the LLMs to leverage explicit indicator features, we first construct a score distribution by concatenating any types of score $\{s_i\}_{i=1}^k$ as features of the top-k retrieved documents and the pre-defined score s_I and s_q for the instruction I and query q as $S = [s_I, s_1, s_2, \dots, s_k, s_q]$. Then, we initialize it by normalizing the feature scores of the retrieved documents $\{s_i\}_{i=1}^k$ to $[0, 1]$ and assign score 1 to the tokens in query and instruction as Eq. 2. The constructed score distribution $S_{\text{norm}} \in \mathbb{R}^{|S| \times |S|}$ is a token-level matrix, i.e., each token has an initial score value. Finally, we incorporate the normalized scores S_{norm} as explicit indicators into the computation of attention networks in *OpenDecoder* modified according to relevance as $\theta_{\text{open}}^{\text{att}}$ via Eq. 3. The intuition is that, by modulating the original attention scores with normalized indicator scores, the importance of each token during the autoregressive decoding would be reshaped to guide the model for answer generation. In extreme cases where all input documents are irrelevant and assigned very low relevance scores, the query and instruction receive relatively higher scores, guiding the model to disregard the retrieved context and instead rely on its parametric knowledge to generate an answer.

$$s_i^{\text{norm}} = \frac{s_i}{\max(\{s_j\}_{j=1}^k)}, \quad s_q^{\text{norm}}, s_I^{\text{norm}} \leftarrow 1 \quad (2)$$

$$S_{\text{norm}} = [s_I^{\text{norm}}, \{s_j^{\text{norm}}\}_{j=1}^k, s_q^{\text{norm}}] \in \mathbb{R}^{|S| \times |S|}$$

Algorithm 1 Modulating LLM internal decoding in *OpenDecoder*

Input: Question q , Relevance score $\{s_i\}_{i=1}^k$ of each input document, Normalization function $\text{Norm}(\cdot)$, Original $\mathcal{LLM}_{\theta_0}$.

Output: Updated $\mathcal{LLM}_{\theta_0+\theta_{\text{open}}^{\text{attn}}}$ and generated answer a .

- 1: Normalize the relevance score among the input documents $\{s_i^{\text{norm}}\}_{i=1}^k = \text{Norm}(\{s_i\}_{i=1}^k)$.
- 2: Construct token-level score matrix $S_{\text{norm}} \in \mathbb{R}^{|S| \times |S|}$ correspond to the input with question q and instruction as Eq. 2.
- 3: Computation of modulated LLM’s internal attention network $\mathcal{LLM}_{\theta_0}$ with external relevance score S_{norm} via new parameter $\theta_{\text{open}}^{\text{attn}}$ as Eq. 3.
- 4: Generate a final answer a for q via the updated $\mathcal{LLM}_{\theta_0+\theta_{\text{open}}^{\text{attn}}}$.

$$\theta_{\text{open}}^{\text{attn}} \sim \text{Attn}(Q, K, V, S_{\text{norm}}) = \text{softmax}\left(\frac{S_{\text{norm}} \cdot QK^T}{\sqrt{d_k}}\right)V \quad (3)$$

The type of scores and normalization approach can be determined according to various criteria such as relevance, reliability, authority, etc. In our implementation, we investigate three types of scores through an aggregation function before the normalization. We expect the relevance score S^{Ret} to be dominant and the other two scores S^{Rank} and S^{QPP} act as supplementary with a scale constant 0.5, which is formulated in Eq. 4.

$$S_{\text{norm}}^{\text{agg}} = \text{Normalize}\left(\text{Aggregate}(S^{\text{Ret}}, S^{\text{Rank}}, S^{\text{QPP}})\right), \text{ where} \quad (4)$$

$$s_{i-\text{agg}}^{\text{norm}} = \frac{(s_{i-\text{Ret}}^{\text{norm}} + 0.5 * (s_{i-\text{Rank}}^{\text{norm}} + s_{i-\text{QPP}}^{\text{norm}}))}{\max(\{s_j^{\text{Ret}} + 0.5 * (s_j^{\text{Rank}} + s_j^{\text{QPP}})\}_{j=1}^k)}, s_{i-\text{agg}}^{\text{norm}} \in S_{\text{norm}}^{\text{agg}}$$

Finally, we optimize to maximize the probability of producing the ground-truth a with the given query and its corresponding retrieved top- k documents set $\{\text{doc}\}_1^k$ as Eq. 5, where θ_0 and $\theta_{\text{open}}^{\text{attn}}$ denote the LLMs’ original parameters and the learned parameters to leverage explicit quality indicator features during fine-tuning, respectively. During inference, the corresponding quality indicator features $\{s_i\}_{i=1}^k$ are required by learned parameters $\theta_{\text{open}}^{\text{attn}}$ for computation of probability in Eq. 3. The core procedure of the information processing within the *OpenDecoder* is described in Algorithm 1.

$$\max_{\theta} \sum_{(q, \{\text{doc}\}_1^k, a)} \sum_{t=1}^{|a|} \log \left(P_{\theta_0+\theta_{\text{open}}^{\text{attn}}} (a_t | a_{<t}, q, \{\text{doc}\}_1^k) \right) \quad (5)$$

3.4 Robustness Training

It may often be the case that some retrieved documents are not relevant. To make the training and inference more robust to noisy information, we conduct robustness training by replacing the second half of the top- k retrieved documents $\{\text{doc}_i\}_{i=1}^k$ with partial relevant ones $\{\text{doc}^{\text{part-rel}}\}$ and irrelevant ones $\{\text{doc}^{\text{irrel}}\}$ as Eq. 6. They are sampled from the top- k set excluding the top-5 documents and the whole collection excluding the top- k documents, respectively. The goal of constructing a noisy document list $\{\text{doc}\}_{\text{noisy}}$ is to provide a necessary environment for the model to learn to distinguish the useful and noisy information. A further alternative is to shuffle the position of the noisy document list as $\{\text{doc}\}_{\text{noisy}}^{\text{shuffle}}$, aiming to emphasize the impact of external signals and reduce the

common issue of position bias [11, 58] of retrieved documents in RAG.

$$\{\text{doc}\}_{\text{noisy}} = \{\text{doc}_i\}_1^5 \cup \{\text{doc}^{\text{part-rel}}\} \cup \{\text{doc}^{\text{irrel}}\}, \quad \text{where} \quad (6)$$

$$\{\text{doc}^{\text{part-rel}}\} \sim \{\text{doc}_i\}_{i=6}^k, \{\text{doc}^{\text{irrel}}\} \sim (C - \{\text{doc}_i\}_{i=1}^k)$$

Then, the reconstructed noisy retrieved documents $\{\text{doc}\}_{\text{noisy}}$ or $\{\text{doc}\}_{\text{noisy}}^{\text{shuffle}}$ with various levels of noise and random relative position are used for robustness training by replacing the original input documents list $\{\text{doc}\}_1^k$ in Eq. 5.

4 Experimental Setup

4.1 Datasets and Evaluation Metrics

We evaluate *OpenDecoder* on five benchmark datasets, including two categories: (1) **General Question Answering**: NQ [24], TriviaQA [19], and PopQA [33], and (2) **Multi-Hop Question Answering**: HotpotQA [57] and 2WikiMultiHopQA [14]. These datasets encompass a diverse range of retrieval with noise in RAG, enabling a comprehensive evaluation in different settings. Statistical details about the used datasets are provided in Appendix A.

4.2 Evaluation Settings in Noisy Environments

We evaluate our *OpenDecoder* and all compared baselines among three settings with different noisy retrieval results. The first one is **Normal Evaluation**, where the input search results for RAG are the original top-10 documents from the retriever. The second one is **Noisy Evaluation**, where the search results for RAG are constructed in the same way as the robust training in Sec. 3.4, i.e., replacing the second half of the top-10 retrieved documents with partial relevant ones and irrelevant ones, which aims to evaluate whether the RAG system can distinguish the noise and solely rely on the useful input information. The third one is **Extreme Noisy Evaluation**, where the search results for RAG are obtained by randomly sampling from the irrelevant document set, which simulates the extreme cases when the retrieval fails among difficult queries or domains.

4.3 Baseline

To evaluate the effectiveness of *OpenDecoder* across various noisy settings, we compare it against the following baselines: (1) Vanilla retrieval-augmented generation (RAG) [25]; (2) Vanilla supervised fine-tuning (SFT) [6]; (3) RobustRAG [54]: An isolate-then-aggregate strategy to filter out the noise in retrieved context; (4) AstuteRAG [48]: A retrieval-refined method to improve knowledge utilization and enhance robustness; (5) InstructRAG [52]: Instructing LLMs to de-noise retrieved content by generating self-synthesized explanatory rationales; (6) Robustness fine-tuning (RbFT) [46]: A more recent approach to conduct robustness training with two instruction fine-tuning tasks, defect detection and utility extraction. More details about the baseline methods can be found in Appendix B.

4.4 Implementation Details

We implement *OpenDecoder* based on Qwen-2.5 series backbone models [56] with the official open-source code repository. The compared baselines are also implemented with the same Qwen-2.5-3B-Instruct model as our main experiments. For retrieval, we use the

Table 1: Main results with three evaluation settings across various noisy environments among the retrieved documents for different RAG systems. To ensure fair and thorough comparison, all methods are based on Qwen-2.5-3B-Instruct backbone models, and the input retrieved documents for each method are fixed to the same. The best and second-best performance is set in bold and underline. [†] and [‡] denote significant improvements with t-test at $p < 0.05$ over the strongest baseline RbFT and the Vanilla SFT without explicit external indicators for training, respectively. [^]/^{*} represents in-domain/out-of-domain datasets.

Evaluation	Method	NQ [^]		TrivialQA [*]		popQA [*]		HotpotQA [^]		2Wiki [*]		Average	
		F1	EM	F1	EM	F1	EM	F1	EM	F1	EM	F1	EM
Normal	No RAG	12.11	-	30.04	-	11.07	-	16.86	-	22.38	-	18.49	-
	Vanilla RAG	25.46	34.12	31.09	48.40	7.35	21.87	12.06	20.73	11.23	20.93	17.44	29.21
	Vanilla SFT	33.63	32.63	50.31	50.53	20.46	17.37	24.02	20.17	19.91	20.33	29.67	28.21
	RobustRAG	26.58	30.80	45.25	47.40	10.91	15.90	13.58	15.73	5.84	9.07	20.43	23.78
	InstructRAG	30.39	31.00	45.81	50.73	16.19	21.70	20.26	22.93	17.37	20.87	26.00	29.45
	AstuteRAG	37.84	34.10	52.28	51.80	23.92	19.90	<u>29.44</u>	23.30	20.79	21.10	32.85	30.04
	RbFT	40.17	36.60	<u>53.49</u>	<u>52.30</u>	<u>24.73</u>	<u>21.42</u>	29.71	24.50	<u>23.02</u>	<u>21.90</u>	<u>34.22</u>	<u>31.34</u>
	<i>OpenDecoder</i>	<u>39.26</u> [‡]	<u>35.90</u> [‡]	56.08 ^{†‡}	54.87 ^{†‡}	25.95 ^{†‡}	22.80 ^{†‡}	<u>29.44</u> [‡]	<u>24.00</u> [‡]	23.63 [‡]	22.53 [‡]	34.87 [‡]	32.02 [‡]
Noisy	Vanilla RAG	15.22	32.70	26.82	49.93	7.83	20.66	11.05	19.00	11.97	20.38	14.58	28.53
	Vanilla SFT	34.98	32.83	48.54	48.07	21.06	18.16	23.55	20.80	22.07	20.40	30.04	28.05
	RobustRAG	25.21	30.20	42.36	44.53	10.33	14.80	12.04	14.13	5.30	8.20	19.05	22.37
	InstructRAG	28.09	29.33	44.13	48.20	14.25	21.40	18.16	11.60	15.30	9.00	23.99	23.91
	AstuteRAG	32.36	29.00	46.81	48.70	20.28	16.60	23.63	17.00	20.84	18.60	28.78	25.98
	RbFT	<u>35.50</u>	<u>30.70</u>	<u>52.62</u>	<u>51.70</u>	<u>23.71</u>	<u>20.20</u>	<u>25.28</u>	<u>19.00</u>	<u>23.60</u>	<u>22.00</u>	<u>32.14</u>	<u>28.72</u>
	<i>OpenDecoder</i>	37.71 ^{†‡}	33.82 [†]	55.09 ^{†‡}	53.33 ^{†‡}	25.07 ^{†‡}	22.02 ^{†‡}	28.76 ^{†‡}	22.77 ^{†‡}	24.17 [‡]	22.13 [‡]	34.16 ^{†‡}	30.81 ^{†‡}
Extreme	Vanilla RAG	3.33	10.14	11.96	18.00	0.98	11.87	4.20	9.67	7.41	13.20	5.58	12.58
	Vanilla SFT	19.78	16.73	34.76	33.40	19.27	18.37	18.26	15.07	21.76	19.93	22.77	20.70
	RobustRAG	3.84	3.93	7.39	7.13	0.39	1.20	1.60	4.67	1.18	3.13	2.88	4.01
	InstructRAG	5.52	7.40	21.51	24.80	1.62	0.70	9.14	5.80	11.25	6.80	9.81	9.10
	AstuteRAG	16.06	9.50	35.03	27.10	15.74	12.80	14.38	10.60	17.36	15.10	19.71	15.02
	RbFT	<u>21.49</u>	<u>17.10</u>	<u>38.18</u>	<u>33.50</u>	<u>21.59</u>	<u>20.80</u>	<u>22.11</u>	<u>15.50</u>	<u>24.28</u>	<u>22.60</u>	<u>25.53</u>	<u>21.90</u>
	<i>OpenDecoder</i>	22.50 ^{†‡}	18.06 ^{†‡}	40.41 ^{†‡}	38.27 ^{†‡}	24.96 ^{†‡}	22.02 ^{†‡}	23.59 ^{†‡}	17.20 ^{†‡}	26.99 ^{†‡}	24.00 ^{†‡}	27.69 ^{†‡}	23.91 ^{†‡}

2018 Wikipedia dump [22] as the knowledge source and E5 [50] as the retriever, with the number of retrieved documents set to 10, following [46, 54]. For the robustness training, the number of relevant, partially relevant, and irrelevant documents is set to the same as the noisy evaluation, as 5, 3, and 2, respectively. The partially relevant and irrelevant documents are randomly sampled five times from corresponding document sets and fixed for all compared methods for fair comparison. For training, we merge the training sets of NQ and HotpotQA to form a unified training dataset for *OpenDecoder* and other fine-tuning-based baselines following [18]. The training epoch is set to 1 to ensure the model learn to use the explicit guidance and generalizes to out-of-domain evaluation datasets without overfitting. Evaluation is conducted on the test sets of five datasets to assess both in-domain and out-of-domain performance. F1 score and Exact Match (EM) are used as the evaluation metrics, following [46, 54]. More implementation details can be found in our public code repository at <https://github.com/fengranMark/OpenDecoder>.

5 Experimental Results

5.1 Main Results

The overall performance of *OpenDecoder* is presented in Table 1. It is tested on five datasets, with three evaluation settings of different noisy environments in terms of the input retrieved documents. We can make the following observations:

(1) Our *OpenDecoder* consistently outperforms most compared baseline methods on three evaluation settings and significantly surpasses the Vanilla SFT approach without external indicators.

Beyond the noisy and extremely noisy evaluation, the retrieved top-k documents in the normal evaluation might still contain noise in the input for answer generation (We will investigate the impact of noise Sec. 5.5). Thus, these results demonstrate the superior effectiveness of our *OpenDecoder* in tolerating noise, which can be attributed to our designed mechanism of modulating the decoding using external relevance signals as indicators and enabling the LLMs to grasp such capacity via specific training.

(2) Compared to other approaches targeting robustness improvement (RobustRAG and RbFT), our *OpenDecoder* exhibits more robust answer generation in noisy and extremely noisy settings. This is mainly because the compared methods still follow the current approach of internal information processing mechanism of the LLMs, which highly rely on the original capacity of the LLMs for distinguishing noise and the bias influenced by system prompts during pre-training. Modulating the LLM decoding with explicit indicators can not only provide useful signals but also alleviate this bias effect.

(3) When the noise in the retrieved document increases, the performance drop is more severe in the relatively simple datasets (NQ and TrivialQA) compared with the other more complex ones (HotpotQA, 2wiki). This means the factoid questions with retrieved support evidence are more sensitive to the input with various noisy levels, thus the external indicators are more useful and necessary; while for the more difficult datasets, the retrieval defects are more common, and thus the urgent goal is to improve the success rate of retrieving relevant documents before aiming to enhance the robustness of the answer generation.

Table 2: Ablation studies on the effectiveness of each mechanism in our *OpenDecoder* training framework.

Method	NQ	TrivialQA	popQA	HotpotQA	2Wiki
Normal Evaluation					
Vanilla SFT	33.63	50.31	20.46	24.02	19.91
w/. Guidance	37.62	55.31	24.33	26.06	20.15
w/. Aggregate	36.24	55.48	21.59	28.86	22.85
w/. Robust Tr.	38.98	55.84	25.14	29.43	22.72
<i>OpenDecoder</i>	39.26	56.08	25.95	29.43	23.63
Noisy Evaluation					
Vanilla SFT	34.98	51.37	21.06	23.55	22.07
w/. Guidance	37.30	53.35	24.05	25.78	23.65
w/. Aggregate	36.42	53.84	23.96	28.39	23.38
w/. Robust Tr.	37.43	54.57	24.56	28.39	23.33
<i>OpenDecoder</i>	37.71	55.09	25.07	28.76	24.17
Extreme Noisy Evaluation					
Vanilla SFT	19.78	34.76	19.27	18.26	21.76
w/. Guidance	21.89	39.03	24.58	20.28	22.79
w/. Aggregate	21.07	39.57	24.26	23.25	26.77
w/. Robust Tr.	22.22	40.33	25.61	23.36	26.52
<i>OpenDecoder</i>	22.50	40.41	24.96	23.59	26.99

5.2 Ablation Study

The ablation studies are shown in Table 2. We can observe that by providing explicit indicators, the LLMs can better process the input information compared to the Vanilla SFT, which is the key idea of our method, and achieve the highest improvement. The feature aggregation is effective in some datasets, while robust training can contribute more to stable performance. A possible explanation is that the most effective features may vary depending on the distribution of the dataset, necessitating a more adaptive feature selection mechanism to enhance generalizability. Meanwhile, introducing noisy training inputs remains essential to improve the model’s robustness and noise tolerance during inference. Nevertheless, combining all these mechanisms for implementing *OpenDecoder* can obtain better results across three different evaluation settings with various levels of noisy context on five datasets, which indicates the effectiveness of each component.

5.3 Feature Aggregation and Normalization

In this section, we further investigate the impact of aggregating and normalizing various scores for answer decoding.

Aggregation. The results of score aggregation are depicted in Figure 3. We can see that aggregating any types of relevant scores can achieve better results compared to the Vanilla SFT without explicit indicators. Leveraging the retrieval score S^{Ret} alone could be sufficient for the general QA datasets (NQ, TrivialQA, and popQA), where aggregating more features might not always bring additional gain. This might be because when one indicator feature is satisfied, adding the others might raise the risk of interference, as these features are measured from different aspects. For the multi-hop QA datasets (HotpotQA and 2wiki), aggregating more feature scores helps to achieve better performance, which implies that complex questions desire more external indications to generate correct answers. In addition, the improvement with aggregating LLM-based ranker score S^{Rank} compared to vanilla SFT demonstrates that the internal information processing of LLMs cannot implicitly ignore

Table 3: The performance using different document position orders in robust training across five datasets.

Method	NQ	TrivialQA	popQA	HotpotQA	2Wiki
Original	35.42	52.57	20.13	20.26	22.07
w/. Reverse	36.39	53.68	21.47	27.91	22.99
w/. Shuffle	37.43	54.57	24.56	28.39	23.33
w/. Noise	37.71	55.09	25.07	28.76	24.17

the noise, which emphasizes the importance of impacting the decoding of LLMs with explicit relevant indicators as our *OpenDecoder*.

Normalization. The results of applying three normalization approaches on aggregating retrieval score S^{Ret} are shown in Figure 4. The Max Normalization is the simplest one, as denoted in Eq. 2. The other two normalization approaches, Min-Max and Exponential-

Rank, are implemented as $\hat{s}_i^{\text{min-max}} = \frac{s_i - \min(\{s_j\}_{j=1}^k)}{\max(\{s_j\}_{j=1}^k) - \min(\{s_j\}_{j=1}^k)}$ and $\hat{s}_i^{\text{Exp}} = \frac{e^{-0.5(i-1)}}{\sum_{j=1}^k e^{-0.5(j-1)}}$, where the former one considers the relative gap among the original scores and the latter one further consider the impact of the rank position with exponential decay for each document candidate. We can observe that the Max normalization performs better than the Min-Max one on general QA datasets, and vice versa on the multi-hop QA datasets. The more complex Exponential normalization with rank decay results in a large performance drop. These observations indicate that applying different normalizations will significantly impact the performance, i.e., appropriate normalization can obtain improvement, while the inappropriate ones would result in a performance drop, even under the same pipeline in our *OpenDecoder*. Thus, a more sophisticated approach could be further explored in future studies.

5.4 Document Order in Robust Training

In this section, we examine the effect of varying document position orders on robust training. As mentioned in Sec. 3.3, the original input context order before applying robust training is $\text{Input} = [\text{Ins}, \text{doc}_1^q, \text{doc}_2^q, \dots, \text{doc}_k^q, \text{q}]$. On top of it, we investigate three types of reorder methods, including reversing the document position from doc_k^q to doc_1^q , shuffling them, and further injecting noise with various relevant levels as Sec. 3.4. The results are presented in Table 3. We observe that reversing the document order can obtain better performance than the original one. This might be because the new reversed order $\text{Input}^{\text{Rev.}} = [\text{Ins}, \text{doc}_k^q, \text{doc}_{k-1}^q, \dots, \text{doc}_1^q, \text{q}]$ enables the higher top- k documents to be much closer to the question and thus might raise their attention score by alleviating the long-distance distraction. This phenomenon suggests that specifying document positions in the prompt template as plain text may not be fully interpreted by LLMs. Consequently, shuffling input documents during training can mitigate position bias, as the top-1 document is not always more informative than the top-2 for answer generation. Moreover, injecting noise further enhances model robustness by encouraging it to assess the true relevance of input documents based on external indicators, rather than relying on positional cues.

5.5 Noise Tolerance of Input Top-K

As the evidence for the correct answer might relate to only a small portion of the relevant documents, the normal evaluation using

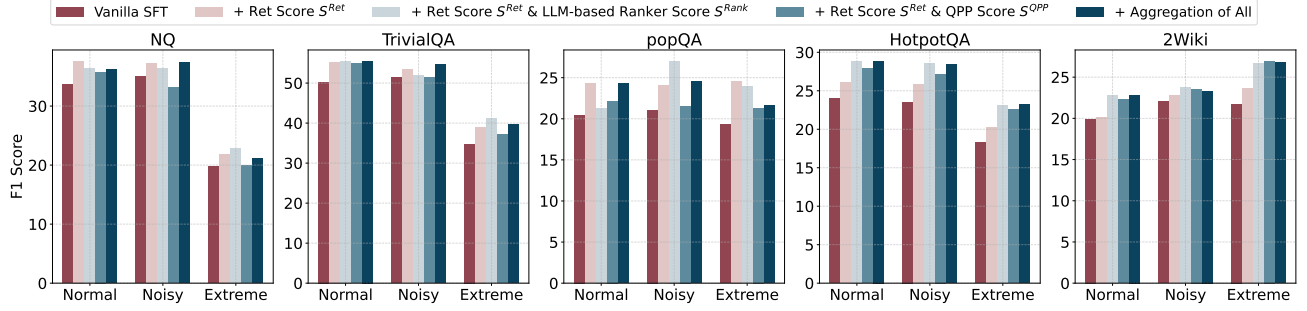


Figure 3: Performance of aggregating various scores as guidance features across different evaluation settings and datasets.

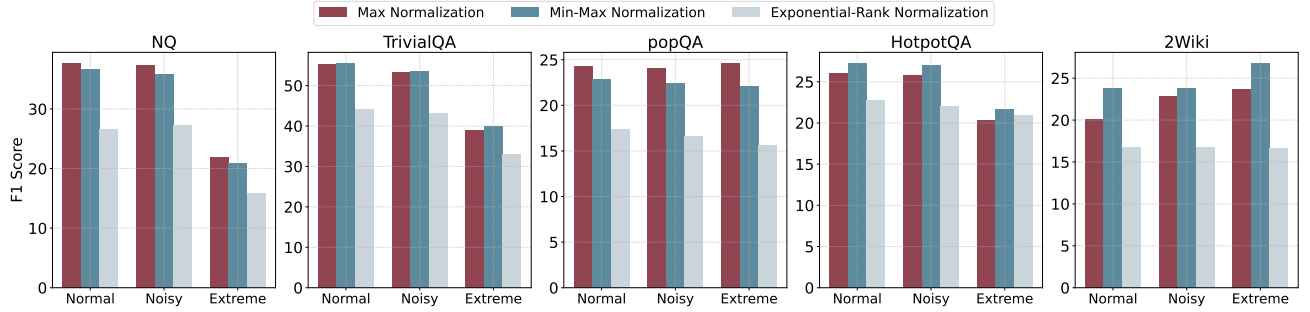


Figure 4: Performance of normalizing scores features with various approaches across different evaluation settings and datasets.

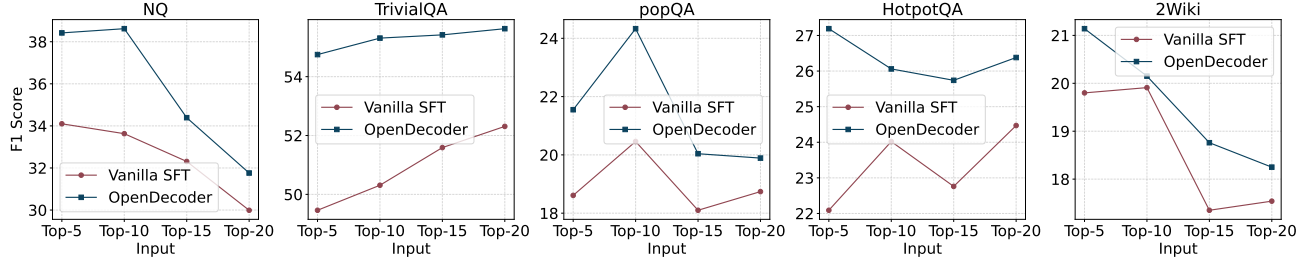


Figure 5: The performance of using various top-k retrieved documents in the normal evaluation setting.

the original top- k retrieved results would still inevitably contain irrelevant information. We evaluate the noise tolerance ability of Vanilla SFT and our proposed *OpenDecoder* in terms of the impact of various input top- k values. The results are shown in Figure 5. As the number of input documents increases, the probability of identifying relevant documents with answer information and the degree of injecting potential noise both increase. In most of the datasets, the larger top- k cannot guarantee higher performance except on TrivialQA, which indicates that the accurate search results are crucial for answer generation. Overall, our *OpenDecoder* exhibits better performance than Vanilla SFT in different numbers of input documents, which demonstrates the effectiveness of leveraging relevance score to impact decoding across various input top- k .

6 Conclusion

In this paper, we propose a new paradigm to modulate the LLMs' internal information processing mechanisms with explicit indicators to improve robustness in answer decoding when the input context contains various noise. To achieve the goal, we proposed

OpenDecoder framework, which constructs various explicit quality indicators via extracting features from the retrieved document and applies them to modify the attention score computation among the networks of LLMs. Additionally, a robustness enhancement mechanism is integrated into the training procedure to enable LLMs to handle various noisy environments. Our experiments demonstrate that incorporating explicit indicators from retrieved information in RAG tasks enhances the LLMs' ability to tolerate noise in the input context and leads to better performance compared to prior approaches. Importantly, this paradigm is flexible to be integrated with the post-training of LLMs for any purposes and incorporated with any type of external indicators.

Acknowledgments

This work was partly supported by a discovery grant from the Natural Science and Engineering Research Council of Canada (NSERC) and the Canada Research Chair on natural language information processing and applications.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [2] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. Self-rag: Learning to retrieve, generate, and critique through self-reflection. In *The International Conference on Learning Representations*.
- [3] Chi-Min Chan, Chunpu Xu, Ruibin Yuan, Hongyin Luo, Wei Xue, Yike Guo, and Jie Fu. 2024. RQ-RAG: Learning to Refine Queries for Retrieval Augmented Generation. In *First Conference on Language Modeling*.
- [4] Chia-Yuan Chang, Zhimeng Jiang, Vineeth Rakesh, Menghai Pan, Chia Michael Yeh, Guanchu Wang, Mingzhi Hu, Zhichao Xu, Yan Zheng, Mahashweta Das, et al. 2024. Main-rag: Multi-agent filtering retrieval-augmented generation. *arXiv preprint arXiv:2501.00332* (2024).
- [5] Krzysztof Marcin Choromanski, Valerii Likhoshesterov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Quincy Davis, Afroz Mohiuddin, Lukas Kaiser, et al. 2021. Rethinking Attention with Performers. In *International Conference on Learning Representations*.
- [6] Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, et al. 2024. Scaling instruction-finetuned language models. *Journal of Machine Learning Research* 25, 70 (2024), 1–53.
- [7] Boyi Deng, Wenjie Wang, Fengbin Zhu, Qifan Wang, and Fuli Feng. 2025. Cram: Credibility-aware attention modification in llms for combating misinformation in rag. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39.
- [8] Qian Dong, Qingyao Ai, Hongning Wang, Yiding Liu, Haitao Li, Weihang Su, Yiqun Liu, Tat-Seng Chua, and Shaoping Ma. 2025. Decoupling Knowledge and Context: An Efficient and Effective Retrieval Augmented Generation Framework via Cross Attention. In *Proceedings of the ACM on Web Conference 2025*.
- [9] Yibing Du, Antoine Bosselut, and Christopher D Manning. 2022. Synthetic disinformation attacks on automated fact verification systems. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 36. 10581–10589.
- [10] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. 2023. Retrieval-Augmented Generation for Large Language Models: A Survey. *preprint arXiv:2312.10997* (2023).
- [11] Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, et al. 2024. A survey on llm-as-a-judge. *arXiv preprint arXiv:2411.15594* (2024).
- [12] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Mingwei Chang. 2020. Retrieval augmented language model pre-training. In *International conference on machine learning*. PMLR, 3929–3938.
- [13] Juyeon Heo, Christina Heinze-Deml, Oussama Elachqar, Kwan Ho Ryan Chan, Shirley You Ren, Andrew Miller, Udhayakumar Nallasamy, and Jaya Narain. 2025. Do LLMs “know” internally when they follow instructions?. In *The Thirteenth International Conference on Learning Representations*.
- [14] Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. 2020. Constructing A Multi-hop QA Dataset for Comprehensive Evaluation of Reasoning Steps. In *Proceedings of the 28th International Conference on Computational Linguistics*. 6609–6625.
- [15] Gautier Izacard, Patrick Lewis, Maria Lomeli, Lucas Hosseini, Fabio Petroni, Timo Schick, Jane Dwivedi-Yu, Armand Joulin, Sebastian Riedel, and Edouard Grave. 2023. Atlas: Few-shot learning with retrieval augmented language models. *Journal of Machine Learning Research* 24, 251 (2023), 1–43.
- [16] Ziwei Ji, Tiezheng Yu, Yan Xu, Nayeon Lee, Etsuko Ishii, and Pascale Fung. 2023. Towards mitigating LLM hallucination via self reflection. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. 1827–1843.
- [17] Bowen Jin, Jinsung Yoon, Jiawei Han, and Sercan O Arik. 2024. Long-context llms meet rag: Overcoming challenges for long inputs in rag. *arXiv preprint arXiv:2410.05983* (2024).
- [18] Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. 2025. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516* (2025).
- [19] Mandar Joshi, Eunsol Choi, Daniel S Weld, and Luke Zettlemoyer. 2017. TriviaQA: A Large Scale Distantly Supervised Challenge Dataset for Reading Comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 1601–1611.
- [20] Minki Kang, Seanie Lee, Jinheon Baek, Kenji Kawaguchi, and Sung Ju Hwang. 2023. Knowledge-augmented reasoning distillation for small language models in knowledge-intensive tasks. *Advances in NeurIPS* (2023).
- [21] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361* (2020).
- [22] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 6769–6781.
- [23] To Eun Kim and Fernando Diaz. 2025. Towards fair rag: On the impact of fair ranking in retrieval-augmented generation. In *Proceedings of the 2025 International ACM SIGIR Conference on Innovative Concepts and Theories in Information Retrieval (ICTIR)*. 33–43.
- [24] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. 2019. Natural questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics* 7 (2019), 453–466.
- [25] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.
- [26] Junyi Li, Jie Chen, Ruiyang Ren, Xiaoxue Cheng, Wayne Xin Zhao, Jian-Yun Nie, and Ji-Rong Wen. 2024. The Dawn After the Dark: An Empirical Study on Factuality Hallucination in Large Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*. 10879–10899.
- [27] Xiaoxi Li, Guanting Dong, Jiajie Jin, Yuyao Zhang, Yujia Zhou, Yutao Zhu, Peitian Zhang, and Zhicheng Dou. 2025. Search-o1: Agentic search-enhanced large reasoning models. *arXiv preprint arXiv:2501.05366* (2025).
- [28] Xiaoqiang Lin, Aritra Ghosh, Bryan Kian Hsiang Low, Anshumali Shrivastava, and Vijai Mohan. 2025. REFRAG: Rethinking RAG based Decoding. *arXiv preprint arXiv:2509.01092* (2025).
- [29] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. 2023. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM computing surveys* 55, 9 (2023), 1–35.
- [30] Yu-An Liu, Ruqing Zhang, Jiafeng Guo, and Maarten de Rijke. 2025. Robust information retrieval. In *Proceedings of the Eighteenth ACM International Conference on Web Search and Data Mining*. 1008–1011.
- [31] Xinbei Ma, Yeyun Gong, Pengcheng He, Hai Zhao, and Nan Duan. 2023. Query rewriting in retrieval-augmented large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 5303–5315.
- [32] Xueguang Ma, Liang Wang, Nan Yang, Furu Wei, and Jimmy Lin. 2024. Fine-tuning llama for multi-stage text retrieval. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*.
- [33] Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi. 2023. When Not to Trust Language Models: Investigating Effectiveness of Parametric and Non-Parametric Memories. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 9802–9822.
- [34] Chuan Meng, Negar Arabzadeh, Arian Askari, Mohammad Aliannejadi, and Maarten de Rijke. 2025. Query performance prediction using relevance judgments generated by large language models. *ACM Transactions on Information Systems* 43, 4 (2025), 1–35.
- [35] Fengran Mo, Kelong Mao, Yutao Zhu, Yihong Wu, Kaiyu Huang, and Jian-Yun Nie. 2023. ConvGQR: Generative Query Reformulation for Conversational Search. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*. 4998–5012.
- [36] Fabio Petroni, Aleksandra Piktus, Angela Fan, Patrick Lewis, Majid Yazdani, Nicola De Cao, James Thorne, Yacine Jernite, Vladimir Karpukhin, Jean Maillard, et al. 2020. KILT: a benchmark for knowledge intensive language tasks. *arXiv preprint arXiv:2009.02252* (2020).
- [37] Hongjin Lin, Zheng Liu, Peitian Zhang, Kelong Mao, Yujia Zhou, Xu Chen, and Zhicheng Dou. 2025. Tackling the Length Barrier: Dynamic Context Browsing for Knowledge-Intensive Task. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1*. 1150–1160.
- [38] Tolga Şakar and Hakan Emekci. 2025. Maximizing RAG efficiency: A comparative analysis of RAG methods. *Natural Language Processing* 31, 1 (2025), 1–25.
- [39] Huatong Song, Jinhao Jiang, Yingqian Min, Jie Chen, Zhipeng Chen, Wayne Xin Zhao, Lei Fang, and Ji-Rong Wen. 2025. R1-searcher: Incentivizing the search capability in llms via reinforcement learning. *arXiv preprint arXiv:2503.05592* (2025).
- [40] Maojia Song, Shang Hong Sim, Rishabh Bhardwaj, Hai Leong Chieu, Navonil Majumder, and Soujanya Poria. 2025. Measuring and Enhancing Trustworthiness of LLMs in RAG through Grounded Attributions and Learning to Refuse. In *The Thirteenth International Conference on Learning Representations*.
- [41] Weihang Su, Yichen Tang, Qingyao Ai, Junxi Yan, Changyue Wang, Hongning Wang, Ziyi Ye, Yujia Zhou, and Yiqun Liu. 2025. Parametric retrieval augmented generation. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1240–1250.
- [42] Weiwei Sun, Lingyong Yan, Xinyu Ma, Shuaiqiang Wang, Pengjie Ren, Zhumin Chen, Dawei Yin, and Zhaochun Ren. 2023. Is ChatGPT Good at Search? Investigating Large Language Models as Re-Ranking Agents. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 14918–14937.
- [43] Zhiwen Tan, Jiaming Huang, Qintong Wu, Hongxuan Zhang, Chenyi Zhuang, and Jinjie Gu. 2025. RAG-R1: Incentivize the Search and Reasoning Capabilities of LLMs through Multi-query Parallelism. *arXiv preprint arXiv:2507.02962* (2025).

- [44] Minghao Tang, Shiyu Ni, Jiafeng Guo, and Keping Bi. 2025. Injecting External Knowledge into the Reasoning Process Enhances Retrieval-Augmented Generation. *arXiv preprint arXiv:2507.19333* (2025).
- [45] Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, Katie Millican, et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805* (2023).
- [46] Yiteng Tu, Weihang Su, Yujia Zhou, Yiqun Liu, and Qingyao Ai. 2025. Robust Fine-tuning for Retrieval Augmented Generation against Retrieval Defects. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1272–1282.
- [47] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [48] Fei Wang, Xingchen Wan, Ruoxi Sun, Jiefeng Chen, and Serkan Ö Arık. 2024. Astute rag: Overcoming imperfect retrieval augmentation and knowledge conflicts for large language models. *arXiv preprint arXiv:2410.07176* (2024).
- [49] Han Wang, Archiki Prasad, Elias Stengel-Eskin, and Mohit Bansal. 2025. Retrieval-augmented generation with conflicting evidence. *arXiv preprint arXiv:2504.13079* (2025).
- [50] Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. 2022. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533* (2022).
- [51] Yuhao Wang, Ruiyang Ren, Yucheng Wang, Wayne Xin Zhao, Jing Liu, Hua Wu, and Haifeng Wang. 2025. Unveiling Knowledge Utilization Mechanisms in LLM-based Retrieval-Augmented Generation. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1262–1271.
- [52] Zhepei Wei, Wei-Lin Chen, and Yu Meng. 2024. InstructRAG: Instructing Retrieval-Augmented Generation via Self-Synthesized Rationales. In *The Thirteenth International Conference on Learning Representations*.
- [53] Orion Weller, Aleem Khan, Nathaniel Weir, Dawn Lawrie, and Benjamin Van Durme. 2024. Defending Against Disinformation Attacks in Open-Domain Question Answering. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics*. 402–417.
- [54] Chong Xiang, Tong Wu, Zexuan Zhong, David Wagner, Danqi Chen, and Prateek Mittal. 2024. Certifiably robust rag against retrieval corruption. *arXiv preprint arXiv:2405.15556* (2024).
- [55] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. Efficient Streaming Language Models with Attention Sinks. In *The Twelfth International Conference on Learning Representations*.
- [56] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).
- [57] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D Manning. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. 2369–2380.
- [58] Jiayi Ye, Yanbo Wang, Yue Huang, Dongping Chen, Qihui Zhang, Nuno Moniz, Tian Gao, Werner Geyer, Chao Huang, Pin-Yu Chen, et al. 2024. Justice or prejudice? quantifying biases in llm-as-a-judge. *arXiv preprint arXiv:2410.02736* (2024).
- [59] Yue Yu, Wei Ping, Zihan Liu, Boxin Wang, Jiaxuan You, Chao Zhang, Mohammad Shoenybi, and Bryan Catanzaro. 2024. Rankrag: Unifying context ranking with retrieval-augmented generation in llms. *Advances in Neural Information Processing Systems* 37 (2024), 121156–121184.
- [60] Zhenrui Yue, Honglei Zhuang, Aijun Bai, Kai Hui, Rolf Jagerman, Hansi Zeng, Zhen Qin, Dong Wang, Xuanhui Wang, and Michael Bendersky. 2025. Inference Scaling for Long-Context Retrieval Augmented Generation. In *The Thirteenth International Conference on Learning Representations*.
- [61] Qinggang Zhang, Zhishang Xiang, Yilin Xiao, Le Wang, Junhui Li, Xinrun Wang, and Jinsong Su. 2025. FaithfulRAG: Fact-Level Conflict Modeling for Context-Faithful Retrieval-Augmented Generation. *arXiv preprint arXiv:2506.08938* (2025).
- [62] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2023. A survey of large language models. *arXiv preprint arXiv:2303.18223* 1, 2 (2023).
- [63] Yuxiang Zheng, Dayuan Fu, Xiangkun Hu, Xiaojie Cai, Lyumanshan Ye, Pengrui Lu, and Pengfei Liu. 2025. Deepresearcher: Scaling deep research via reinforcement learning in real-world environments. *arXiv preprint arXiv:2504.03160* (2025).
- [64] Huichi Zhou, Kin-Hei Lee, Zhonghao Zhan, Yue Chen, and Zhenhao Li. 2025. Trustrag: Enhancing robustness and trustworthiness in rag. *arXiv e-prints* (2025).
- [65] Yujia Zhou, Yan Liu, Xiaoxi Li, Jiajie Jin, Hongjin Qian, Zheng Liu, Chaozhao Li, Zhicheng Dou, Tsung-Yi Ho, and Philip S Yu. 2024. Trustworthiness in retrieval-augmented generation systems: A survey. *arXiv preprint arXiv:2409.10102* (2024).

Appendix

A Datasets Details

Table 4: Statistics of the five used datasets.

	NQ	TrivialQA	popQA	HotpotQA	2Wiki
#Train Q	79,168	-	-	90,447	-
#Test Q	3,610	11,312	1,399	7,405	9,322
#Collection			21M		

We use five benchmarks for evaluation, and the unified training set from NQ and HotpotQA to fine-tune our *OpenDecoder*. The statistics of the used datasets are presented in Table 4 and their detailed description are shown below:

- **NaturalQuestion (NQ)** is a factoid dataset whose questions consist of real anonymized, aggregated queries issued to the Google search engine.
- **TrivialQA** is a reading comprehension dataset whose question-answer pairs authored by trivia enthusiasts and independently gathered evidence documents that provide high quality distant supervision for answering the questions.
- **PopQA** assesses factual question answering, challenging the model’s ability to recall accurate knowledge and resolve ambiguity in entity representation.
- **HotpotQA** focuses on evaluating multi-hop reasoning skills, requiring models to combine information from different contexts to address a single query.
- **2WikiMultihopQA (2wiki)** is a dataset designed to test the model’s ability to perform multi-hop reasoning by integrating information across multiple Wikipedia passages.

B Baseline Details

All compared baselines are implemented by us using the same retrieved document sets across evaluation settings to guarantee fairness in comparison. The instruction used for Vanilla RAG, Vanilla SFT, and our *OpenDecoder* is the same as “You should answer the question by referring to the retrieved knowledge provided below and integrating the usefulness of your own parametric knowledge. Just directly answer it as a short answer without any explanation.” For the prompting-based methods, RobustRAG, InstructRAG, and AstuteRAG, we inherit their original instruction provided in the corresponding code repository. For the fine-tuning-based methods RbFT, we also use its original instruction, but set the same hyperparameter as *OpenDecoder*.

C Investigation of Scaling Model Size

We further investigate the impact of scaling up model size for vanilla SFT and our *OpenDecoder*. The results in the noisy evaluation setting are depicted in Figure 6. Overall, both the SFT and our proposed approaches benefit from larger model sizes, suggesting that larger models are more capable of tolerating contextual noise, which aligns with prior studies [21]. Moreover, the effectiveness of leveraging explicit indicators to influence answer generation becomes more pronounced with larger models, whereas smaller

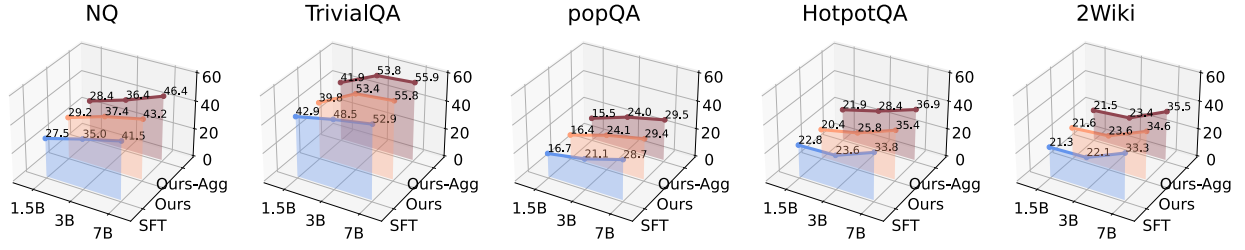


Figure 6: Comparison between SFT and *OpenDecoder* of scaling model size across five datasets in the noisy evaluation setting.

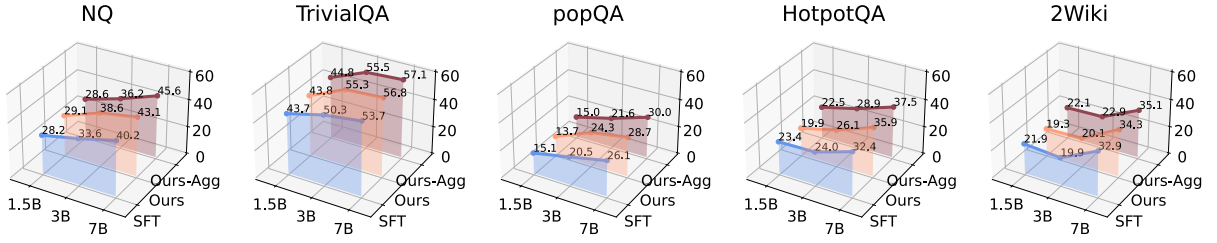


Figure 7: Comparison between SFT and *OpenDecoder* of scaling model size across five datasets in the normal evaluation setting.

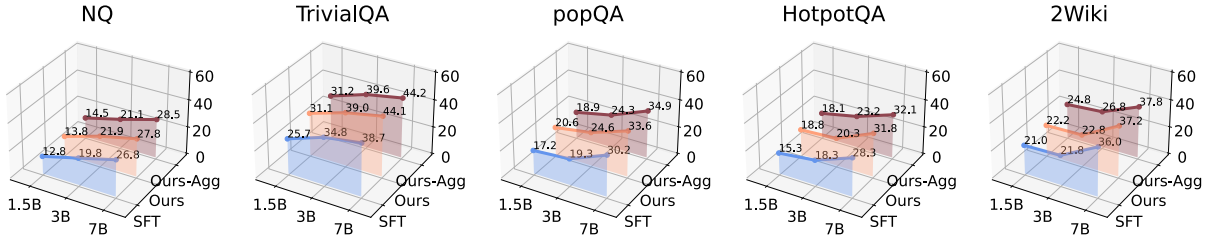


Figure 8: Comparison between SFT and *OpenDecoder* of scaling model size across five datasets in the extreme noisy evaluation setting.

models (e.g., 1.5B) do not consistently achieve better performance across all datasets. These observations indicate that effectively integrating external signals with internal LLM reasoning processes is a non-trivial task that demands higher model capacity. A similar trend is observed when aggregating multiple guidance score features, implying that this aggregation process also requires implicit learning during training.

The results in the normal evaluation and extreme noisy setting are also depicted in Figure 7 and Figure 8, respectively. Overall, similar trends are observed in the noisy evaluation setting, where larger models are more capable of tolerating contextual noise. Besides, the improvement in scaling model size is more pronounced in complex QA datasets than in general ones, indicating that a larger model may be equipped with a more powerful reasoning ability implicitly. Therefore, designing more sophisticated learning objectives to better incorporate this aggregation mechanism and employing larger backbone models for training *OpenDecoder* could further enhance performance, which we leave for future work.

D Discussion on Time and Space Efficiency

The computation cost of our method is the same for the offline training and online inference. The computation complexity of the Vanilla SFT method and our *OpenDecoder* are $O(|d|^2h + |d|h^2)$ in the RAG setting, where d is the average number of tokens in a document doc, and h is the hidden dimension size of the decoder-only LLMs. This is because the explicit guidance, i.e., the relevance scores, are produced simultaneously with the retrieved documents, and the normalization of the scores should be negligible. In terms of the storage overhead, the normalized score $S_{\text{norm}} \in \mathbb{R}^{h \times h}$ is stored as a token-level metric, whose shape is the same as the Query, Key, and Value metric in the attention computational network inside the LLMs. Thus, the additional storage overhead compared to Vanilla SFT is $O(nh)$, where n is the number of Transformer layers with the impact of explicit guidance.